Master Sciences pour l'Ingénieur

UE Programmation C++

Projet — Générateur de Donjon et Labyrinthe Interactif

10 avril 2026

1 Informations Générales

Équipe	Travail en binôme (2 personnes)
Durée	2–3 semaines
Langage	C++
Plateforme	Linux / Windows / macOS
IDE	VSCode ou tout autre environnement

Objectif : Concevoir et implémenter un générateur de donjon/labyrinthe interactif en ligne de commande, explorant les concepts clés du cours C++ : classes, héritage, polymorphisme, structures de données et design patterns.

2 Contexte et Concepts Clés

Vous allez créer un système de génération de donjon/labyrinthe composé des éléments suivants :

- Un donjon généré aléatoirement avec des couloirs et des cases spéciales
- Différents types de cases : murs, passages, trésors, monstres, pièges
- Un aventurier qui explore le donjon
- Affichage ASCII du donjon en temps réel
- Calcul du chemin optimal pour sortir du donjon (BFS)
- Application du **design pattern Factory** pour la création des cases

3 Exigences Fonctionnelles

3.1 Structure Objet — Classes et Héritage

Un labyrinthe est composé de case, de différents types.

Vous **devez** implémenter au minimum la hiérarchie suivante, complétée par la `CaseFactory` (voir section 3.3) :

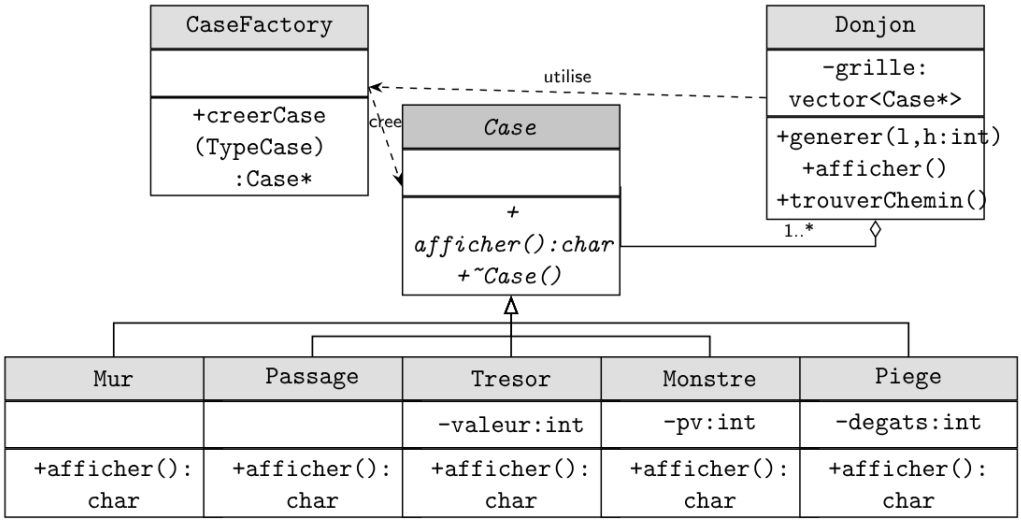


FIGURE 1 – Diagramme de classes UML. `Case` est abstraite (italique) ; ses cinq sous-classes concrètes l'implémentent. `CaseFactory` centralise l'instanciation (patron Factory). `Donjon` agrège des `Case*`.

```
// Classe abstraite de base
class Case {
public:
    virtual char afficher() = 0;
    virtual ~Case() = default;
};

class Mur     : public Case { /* ... */ };
class Passage : public Case { /* ... */ };
class Tresor  : public Case { /* ... */ };
class Monstre : public Case { /* ... */ };
class Piege   : public Case { /* ... */ };

class Donjon {
    vector<vector<Case*>> grille;
public:
    void generer(int largeur, int hauteur);
    void afficher();
    vector<pair<int,int>> trouverChemin(/* ... */);
};
```

3.2 Affichage Console — Exemple

Le donjon s'affiche en ASCII. L'aventurier est représenté par `@`, les trésors par `+`, les monstres par `M`, les pièges par `T` et les murs par `#`.

```
+------------------------+
|## ### # # #####        |
|#  #   #   #   # +    M |
|# ### ### # ##### ##### |
|#   #   #   #   #       |
|##### ### ### ###  # #  |
|   #   #   #   #   # #T |
|##### ### ###  # # ### #|
|       #   #           #|
| +   ######### ######## |
|       #    #@#          |
+------------------------+

Position    : (12, 9)
Sante       : 100/100
Inventaire  : 2 tresors
Distance a la sortie : 23 cases
```

3.3 Design Pattern — Factory

Sans pattern, le code de génération du donjon serait truffé de `new Mur()`, `new Tresor()`... dispersés dans plusieurs méthodes. L'ajout d'un nouveau type de case (ex. `Porte`) imposerait de modifier tous ces endroits.

Avec la **Factory**, **un seul endroit centralise la création** : `CaseFactory::creerCase()`. Le `Donjon` ne connaît que l'enum `TypeCase` — jamais les classes concrètes. Un enum est un type particulier qui permet de définir un ensemble fini de valeurs possibles. Par exemple, `TypeCase` peut représenter les différents types de cases du donjon, comme MUR, SOL, PORTE, etc. L'intérêt est de manipuler des valeurs symboliques claires plutôt que des chaînes de caractères ou des entiers.

Principe

Le pattern Factory consiste à déléguer la création d'objets à une classe dédiée, la `CaseFactory`, plutôt que d'instancier directement les sous-classes partout dans le code. La factory expose une unique méthode statique `creerCase(TypeCase)` qui prend en paramètre un type énuméré et retourne un pointeur vers la case correspondante.

Concrètement, dans notre projet, cela signifie que **ni `Donjon` ni l'algorithme de génération ne connaissent les classes concrètes** (`Mur`, `Tresor`, etc.). Ils ne manipulent que l'enum `TypeCase` et des `Case*`. L'implémentation de la factory sera à votre charge : vous devrez définir l'enum, la classe `CaseFactory` et sa méthode de création.

Utilisation dans `Donjon`

La `CaseFactory` est appelée à deux moments clés de la vie du donjon :

— **Lors de l'initialisation** : toute la grille est d'abord remplie de cases de type `MUR` en appelant `creerCase(TypeCase::MUR)` pour chaque cellule.
— **Lors du placement des éléments** : une fois les passages creusés par l'algorithme de génération, certaines cases de type `PASSAGE` sont remplacées aléatoirement par des trésors, monstres ou pièges via des appels successifs à `creerCase(...)`.

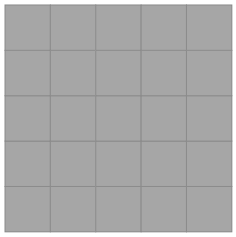
Ainsi, si vous souhaitez ajouter un nouveau type de case (par exemple une `Porte`), il vous suffit d'étendre l'enum et la factory — le reste du code n'a pas à être modifié.

Avantages du pattern Factory dans ce projet :

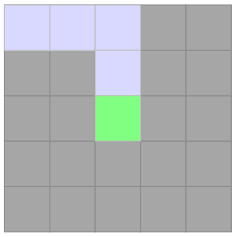
— **Découplage :** `Donjon` ne dépend pas des classes concrètes
— **Extensibilité :** ajouter un nouveau type de case ne touche qu'à `CaseFactory` et à l'enum
— **Lisibilité :** `creerCase(TypeCase::TRESOR)` est plus expressif qu'un `new` isolé
— **Testabilité :** on peut mocker la factory pour tester la logique de génération indépendamment

3.4 Génération du Donjon

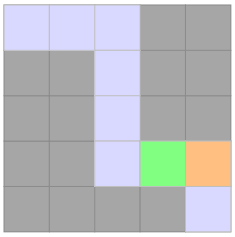
Le donjon est une grille de taille paramétrable (ex. 20×20). L'algorithme de **Recursive Backtracking** (parcours en profondeur avec retour arrière) sera utilisé pour générer des donjon cohérent. Il garantit un labyrinthe parfait — il existe exactement un chemin entre deux cases quelconques. La figure 2 présente une étape de backtracking de l'algorithme.



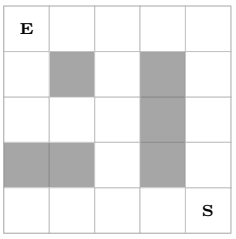
(a) Grille initiale



(b) Avance en profondeur



(c) Impasse, retour arrière



(d) Labyrinthe terminé

Légende

FIGURE 2 – Génération de labyrinthe par *Recursive Backtracking*. (a) Grille initiale tout en murs. (b) L'algorithme creuse des passages récursivement depuis la case de départ. (c) Face à une impasse (tous les voisins sont déjà visités), il revient en arrière (*backtrack*) jusqu'à trouver un voisin non visité. (d) Résultat final : un labyrinthe parfait où il existe exactement un chemin entre deux cases quelconques.

Le pseudo code de l'algorithme est le suivant :

Listing 1 – Pseudo-code — Recursive Backtracking

```
fonction genererLabyrinthe(grille, x, y):
    marquer grille[x][y] comme visite
    directions = [NORD, SUD, EST, OUEST]
    melanger(directions) // aleatoire

    pour chaque direction d dans directions:
        (nx, ny) = voisin de (x,y) dans direction d (2 cases)
        si (nx,ny) dans bornes ET non visite:
            casser le mur entre (x,y) et (nx,ny)
            genererLabyrinthe(grille, nx, ny)  // recursif

fonction initialiserGrille(largeur, hauteur):
    pour chaque case (i,j):
        grille[i][j] = CaseFactory::creerCase(TypeCase::MUR)
    genererLabyrinthe(grille, 1, 1)
    poserEntree(grille)   // ex: coin haut-gauche
    poserSortie(grille)   // ex: coin bas-droit

fonction placerElements(grille):
    pour chaque case (i,j) de type PASSAGE:
        r = nombre_aleatoire(0, 100)
        si r < 5  : grille[i][j] = CaseFactory::creerCase(TRESOR
           )
        sinon si r < 10 : grille[i][j] = CaseFactory::creerCase(
           MONSTRE)
        sinon si r < 13 : grille[i][j] = CaseFactory::creerCase(
           PIEGE)
```

Les éléments à respecter :
- Taille paramétrable (ex. 20×20)
- Utiliser `CaseFactory` pour instancier chaque case (voir section 3.3)
- Garantir l'existence d'un chemin de l'entrée à la sortie
- Placer aléatoirement : trésors, monstres, pièges

3.5 Aventurier et Exploration

La classe `Aventurier` encapsule l'état du joueur : sa position dans le donjon, ses points de vie et son inventaire (les trésors ramassés). Elle est le point central autour duquel s'articule la boucle de jeu.

Boucle de jeu

La boucle principale tourne tant que l'aventurier est en vie et n'a pas atteint la sortie. À chaque itération, on affiche l'état du donjon et les statistiques du joueur, on lit une touche clavier (par exemple `z/q/s/d` pour les quatre directions, `p` pour demander le chemin optimal), on calcule la position cible et on applique l'effet de la case atteinte.

Si la case cible est un **mur**, le déplacement est simplement refusé. Pour toute autre case, l'aventurier se déplace et l'effet est résolu : ramassage d'un trésor, rencontre d'un

monstre (avec choix combat ou fuite), ou déclenchement d'un piège. Lorsque les points de vie tombent à zéro, la partie se termine.

Fonctions à implémenter

Vous êtes libres de structurer votre implémentation comme vous le souhaitez. Voici les spécifications des fonctions attendues ; c'est à vous de choisir dans quelles classes les placer et comment les relier :

- `deplacer(int nx, int ny)` — Met à jour la position de l'aventurier. Précondition : la case `(nx, ny)` est franchissable (pas un mur). Ne gère pas les effets de la case.
- `resoudreCase(Case* c)` — Applique l'effet de la case sur laquelle l'aventurier vient d'arriver : ajout au trésor, perte de points de vie, interaction avec un monstre. La case peut être modifiée (ex. trésor remplacé par un passage après ramassage).
- `estVivant() : bool` — Retourne `true` tant que les points de vie sont strictement positifs.
- `afficherStatut()` — Affiche en console la position courante, les points de vie et le contenu de l'inventaire.
- `boucleDeJeu(Donjon& d)` — Orchestre l'ensemble : affichage, lecture clavier, déplacement, résolution de case et vérification des conditions de fin de partie.

Fonctionnalités attendues :

- Déplacement dans les 4 directions (haut, bas, gauche, droite)
- Ramasser les trésors (mise à jour de l'inventaire)
- Rencontrer des monstres : combat simple ou fuite
- Affichage de l'inventaire et de l'état de santé

3.6 Pathfinding — BFS

L'algorithme BFS (*Breadth-First Search*) garantit de trouver le chemin **le plus court** en nombre de cases sur une grille non pondérée.

Conseil : testez votre BFS sur papier sur une grille 5×5 avant de coder. Tracez la file et les cases visitées étape par étape — cela évite la plupart des bugs d'index.

Listing 2 – Pseudo-code — BFS avec reconstruction du chemin

```
fonction trouverChemin(grille, depart, arrivee):
    file    = file_vide()
    visite  = tableau 2D de booleens, tout a FAUX
    parent  = tableau 2D de positions, tout a (-1,-1)

    enfiler(file, depart)
    visite[depart] = VRAI

    tant que file non vide:
        courant = defiler(file)

        si courant == arrivee:
            retourner reconstruireChemin(parent, depart, arrivee
               )

        pour chaque voisin v de courant (4 directions):
            si v dans bornes ET non visite ET v n'est pas MUR:
                visite[v] = VRAI
                parent[v] = courant
                enfiler(file, v)

    retourner []  // aucun chemin trouve

fonction reconstruireChemin(parent, depart, arrivee):
    chemin  = liste vide
    courant = arrivee

    tant que courant != depart:
        ajouter courant en tete de chemin
        courant = parent[courant]

    ajouter depart en tete de chemin
    retourner chemin

// Affichage : marquer les cases du chemin avec '.' sur une
   copie
// de la grille, puis afficher distance = longueur(chemin) - 1
```

4 Exigences Techniques C++

- Classes et héritage polymorphe (`virtual`)
- Design pattern **Factory** (`CaseFactory`, obligatoire)
- Surcharge d'opérateurs (« pour afficher, par exemple)
- Structures de données de la librairie standard : `vector`, `map`, `queue` (pour BFS)

5 Fonctionnalités Optionnelles (Bonus)

- Autres algorithmes de génération de labyrinthe (à expliciter dans le rapport)

— Sauvegarde/Chargement du donjon dans un fichier
— Système de combat avec monstres (dés, probabilités)
— Plusieurs aventuriers qui collaborent
— Statistiques : temps d'exploration, nombre de cases visitées

6 Livrables

— **Code source complet**
— **Rapport PDF** (3 pages) :
  — Architecture globales et description des classes principales
  — Algorithmes utilisés : justification des choix d'implémentation et des structures de données utilisées.
  — Difficultés rencontrées et solutions
— **Exemple de donjons** générés et sauvegardés (dans des fichiers .txt)
— **Journal de prompts** si utilisé.

Le projet est à rendre sur moodle pour le **7 Mai, 20h.** Un seul rendu est attendu par binôme. Mentionnez bien vous deux noms dans le rendu.

7 Critères d'Évaluation

Critère	**Pondération**
Architecture et Design OOP	20%
Design Pattern Factory	10%
Génération du labyrinthe	20%
Exploration et mouvements	15%
Pathfinding (BFS)	15%
Affichage et UX console	10%
Qualité du code et normes	10%
Total	**100%**
Bonus et fonctionnalités extra	+5%

Bon courage !